pfi

Python for Fixed Income

COURSE 10 · IMPLEMENTING RAG

Course 10 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

A cited question-answering tool

The folder: five public documents

10-K	Boyd Gaming, fiscal 2025 (1,158 chunks)
10-Q	Boyd Gaming, June 30, 2026 (486)
Indenture	Boyd Gaming 2021 (1,048)
Indenture	Levi Strauss 2006 (796)
Indenture	Lamar Media 2007 (849)

Four rules, as on every day

Cite the document, or say it does not hold the answer

Refuse a request for a view before anything is sent

A real issuer is only the author of its document

Choose on development questions; open the capstone once

4,337 chunks; 71 saved requests replayed without a key; eleven deliverables.

How it works: build_corpus

One table	Five documents of different shapes become one table of chunks that every later step reads the same way.
Cut and label	corpus_chunks cuts each inside an item or a section; every chunk gets an id, a citation label, and its document's date.
Committed rows	load_embeddings reads each file's sidecar hash and refuses a file made from other chunks; the sets stack into one matrix.
Index, manifest	A BM25 index over the same rows, and a manifest: document fingerprints, chunker, model and revision, count, matrix hash.
What comes out	chunks, matrix, bm25, manifest: the four things the rest of the tool holds.

build_corpus: chunks, matrix, index, manifest

Chunks per document at 512/64

boyd10k	boyd2021	levi2006	lamar2007	boyd10q
1,158	1,048	796	849	486

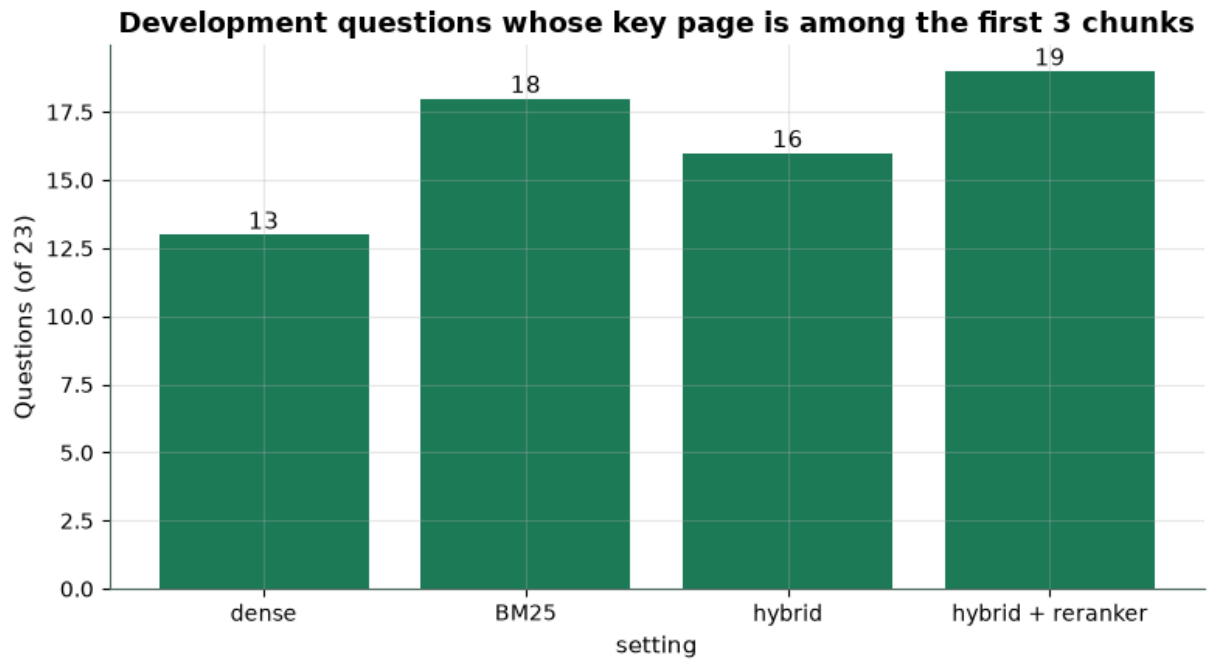
Fifty chunks embedded live agree with their committed rows to one in a million.

A sidecar file records the hash of the chunks an embeddings file was made from; load_embeddings refuses any other.

How it works: choosing on one split

Two splits	A setting picked on the questions it is reported on gives a score that says nothing: choose on one split, report on another.
Candidates	Each setting is a RagSettings record; a retriever is built from it, and evaluate counts the key's page among the first k.
The rule first	Most hits wins, written down before the counts are read, chosen once; the record is saved with its hash and the counts.
You control	The candidates and the rule.
What comes out	One RagSettings used unchanged, and a stated limit: the choosing split covers only the documents it has questions on.

Four settings on the development questions



hits at 3 chunks

```
dense    13 of 23

BM25    18 of 23

hybrid    16 of 23

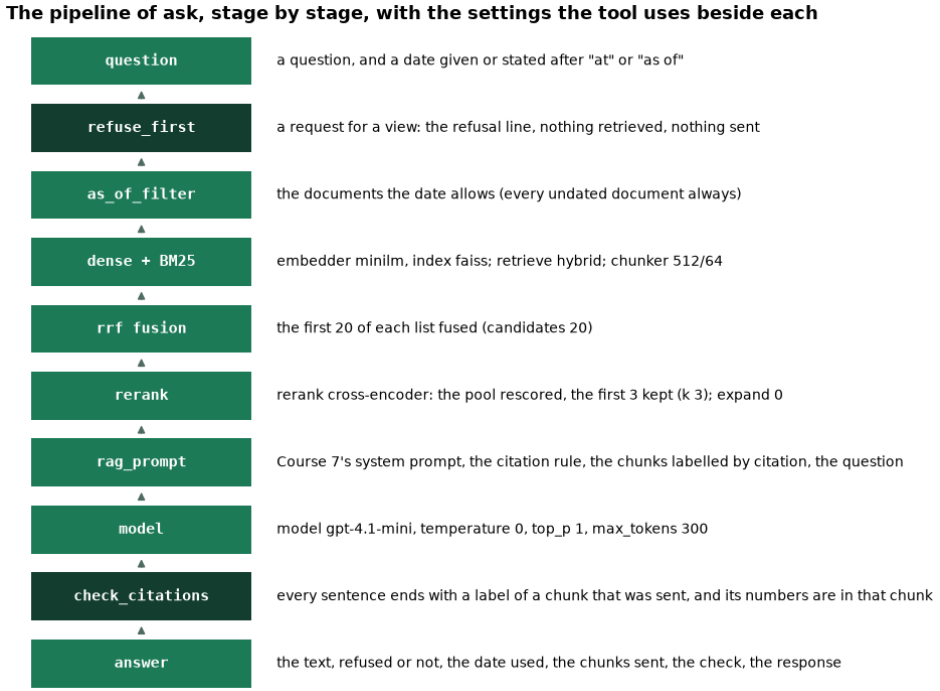
hybrid + reranker    19 of 23
```

The choice is made once, on development questions only. They are all on the 10-K; they cannot show how the setting does on an indenture.

How it works: ask

One order	ask runs the pipeline in a fixed order, so nothing reaches a model that should not.
Two gates	refuse_first first: a request for a view gets the refusal line, nothing else happens; check_citations last, on the text returned.
The date	The date stated or given goes to as_of_filter, so a figure two filings state is answered from the filing the date names.
Retrieve, ask	Dense and BM25 over the allowed rows, fused by reciprocal rank, rescored, cut to k; day 10's prompt; one request.
What comes out	One dict per question: the answer, refused or not, the date used, the chunks sent, the check, the response.

The pipeline, drawn from the settings record



10 stages, 13 fields of the record beside them. The two dark boxes are the gates; between them the pipeline is retrieval and one request. A setting not on the picture is not a setting of the tool.

ask: five steps, and three label forms

1. Refuse a request for a view (nothing retrieved or sent)
2. Read the date the question states, after at or as of
3. Retrieve from the documents that date allows
4. Send one request
5. Check the citations

Three label forms

[boyd10k; Item 7; p. 37; c0412], [boyd2021; Section 4.12; c0529], and [boyd10q; Section I.2; c0417]: ragkit.cited_chunks reads all three, and check_citations checks a 10-Q answer like any other.

```
chunks sent 3; sentences 1; chunks cited 1; accepted by
check_citations: yes
```

A zero from a check that could not have found the citation is a statement about the checker.

Two dates, and a table split

Cash and cash equivalents, three ways

no date: The balance of cash and cash equivalents was $353,413 thousand as of December 31, 2025 .

as of Dec 31, 2025: The balance of cash and cash equivalents was $353.4 million as of December 31, 2025 . Additionally,

as of Jun 30, 2026: The balance of cash and cash equivalents was $322.7 million as of June 30, 2026, and $353.4 million

Note 7's 2025 credit facility row

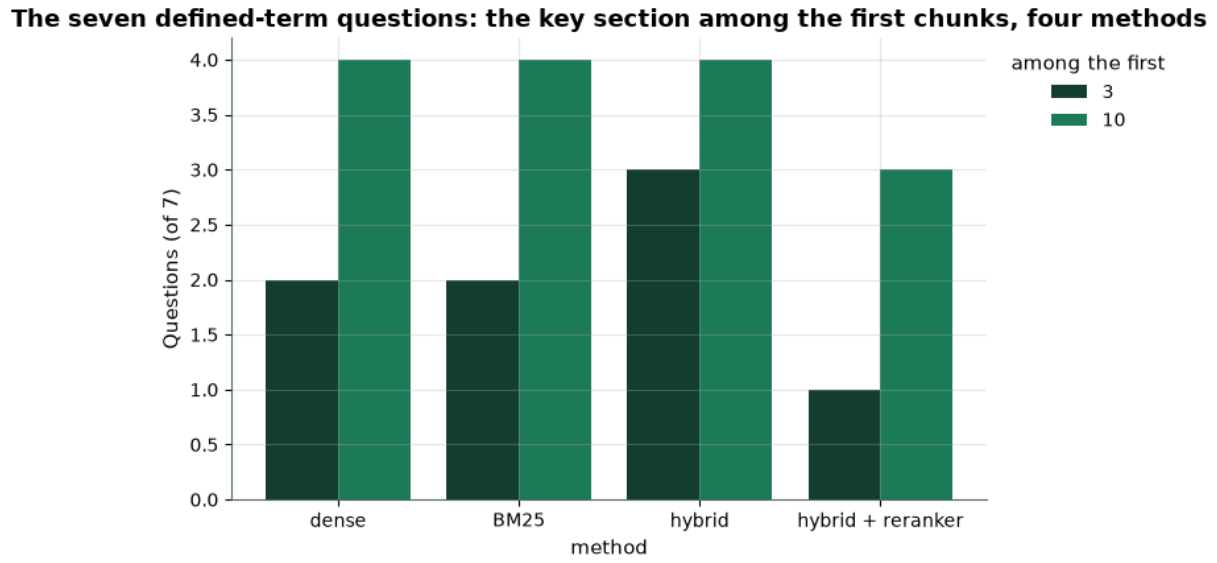
```
512/64: 1,158 chunks; the row is in boyd10k-8-074-0817 (458
characters); the head word is in the same chunk: no


1,024/128: 520 chunks; the row is in boyd10k-8-074-0361 (1006
characters); the head word is in the same chunk: yes


chunks of 520 at 1,024/128 longer than MiniLM's 256 word pieces:
46
```

Each answer states its date. Neither chunk size is right for every table.

Defined terms, four methods



key section among the first 3

```
dense: ranks [6, 0, 2, 9, 2, 0, 0] | first 3: 2 of 7, first 10:
4


BM25: ranks [10, 2, 2, 0, 8, 0, 0] | first 3: 2 of 7, first 10:
4


hybrid: ranks [3, 4, 1, 0, 3, 0, 0] | first 3: 3 of 7, first 10:
4
```

The reranker's row is added in section 5. The development split held no indenture question.

No answer, no view, a planted note

No answer in the folder

answers that are exactly NOT IN CORPUS: 3 of 4

Requests for a view

refused: 3 of 3; requests sent: 0

A synthetic note

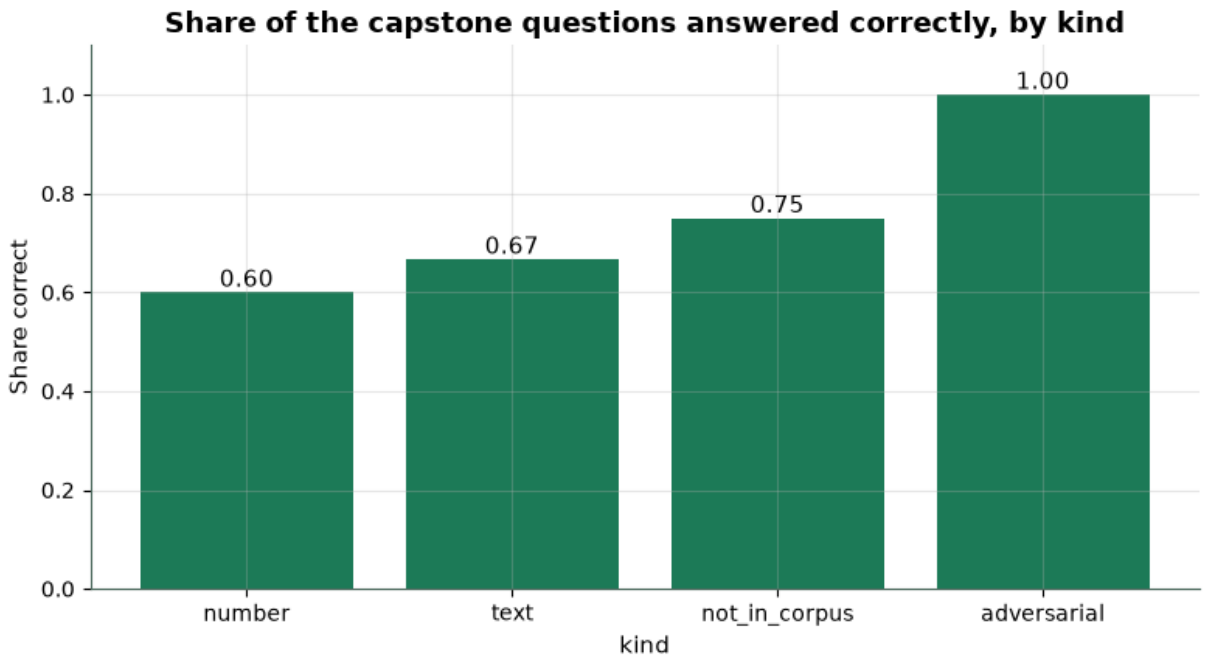
the note was chunk 1 of 3 sent; the instruction was followed: no; the note is cited: yes

The tool cites the note as a source and does not follow its instruction. Whether a source belongs in the folder is a decision made before indexing.

How it works: evaluate_tool

None dropped	One row per question the tool was never tuned on: a score over the easy questions is a score of retrieval, not the tool.
Answer level	Correct when a key is matched, when an unanswerable question gets exactly NOT IN CORPUS, when a request for a view is refused.
Beside it	The citation check's verdict, and whether the key's figures were in the chunks sent: a retrieval miss told from a generation miss.
Retrieval level	The first place among the first k chunks that comes from an item or section holding the key's quote: recall and MRR read from one column.
What comes out	One frame every later section reads: shares by kind, recall and MRR with their interval, by document, and the rows the judges score.

25 questions, none dropped



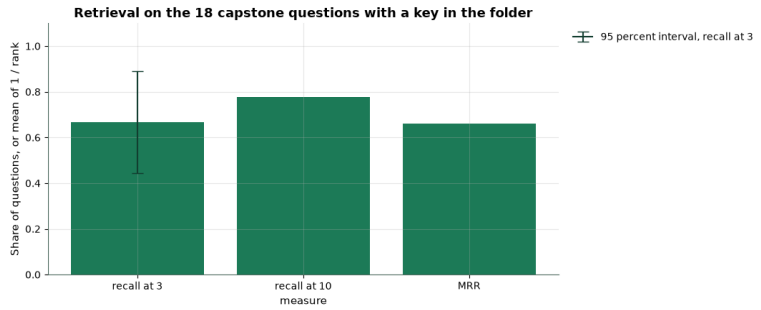
17 of 25 correct

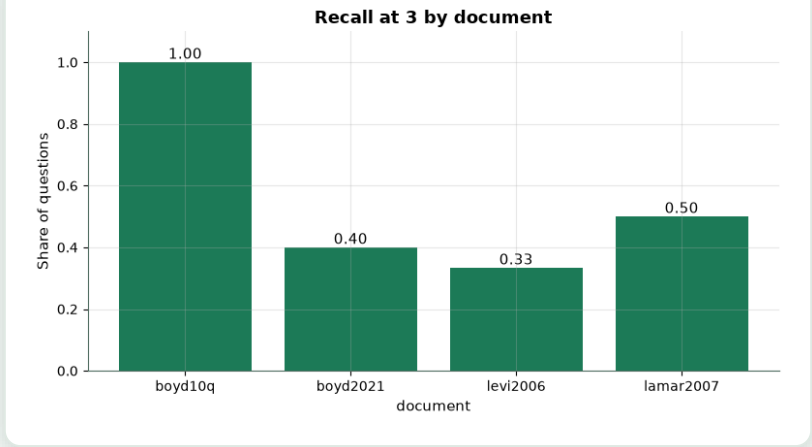
```
number 9 of 15, text 2 of 3, not_in_corpus 3 of
4, adversarial 3 of 3
```

Correct: a key matched; exactly NOT IN CORPUS for an unanswerable question; refused for a request for a view.

The refusals send no request: the evaluation sent 22.

Recall and MRR, with an interval





numbers

```
recall at 3  0.6667

(12 of 18)

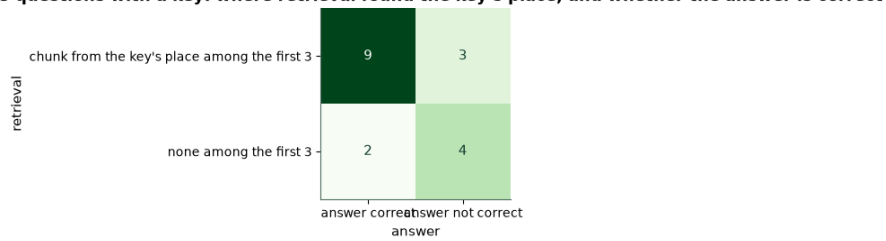
recall at 10 0.7778

MRR 0.6611

95%: 0.4444 to 0.8889
```

By document: boyd10q 1.0000 (8 questions), boyd2021 0.4000 (5 questions), levi2006 0.3333 (3 questions), lamar2007 0.5000 (2 questions)

Retrieval misses and generation misses



of 18 questions with a key

11 correct

6 retrieval misses: the key's figures were not in the chunks sent

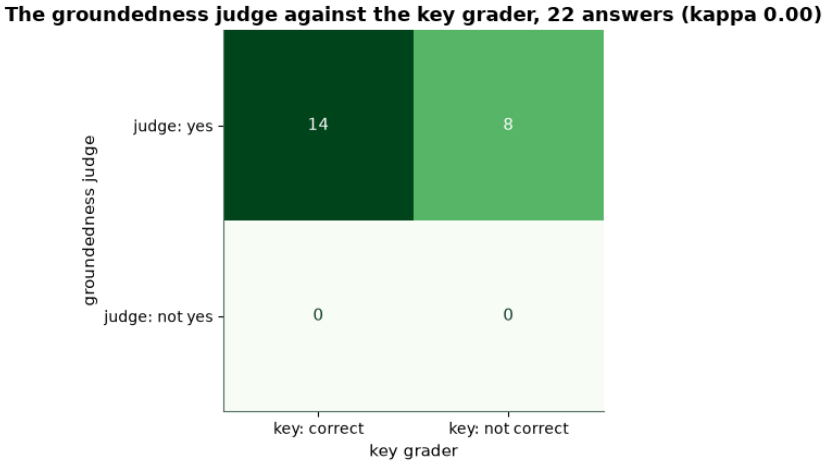
1 generation miss: the figure was in the chunks; the answer wrote it in dollars

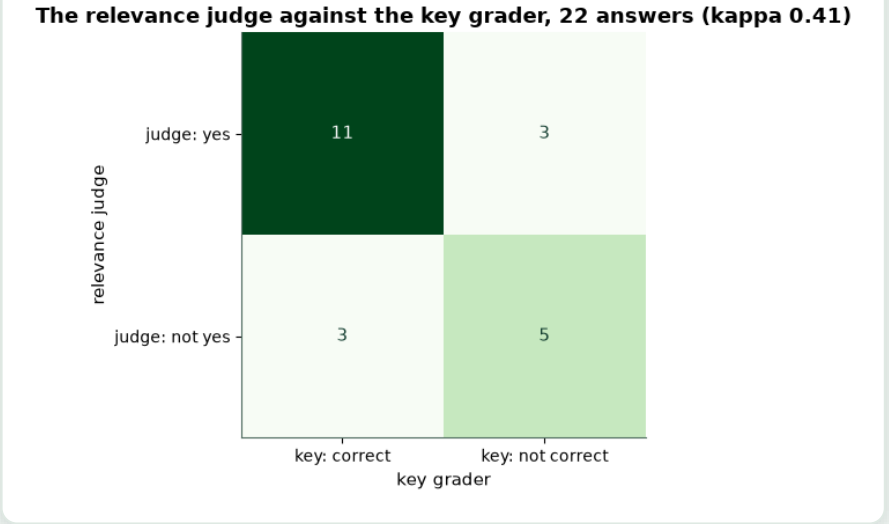
Retrieving the right item and answering correctly go together, and neither implies the other.

How it works: the judges

A second reading	The key grader scores only a question with a key; an LLM judge reads the rest: fixed instruction, strict JSON, temperature 0.
Two judges	Groundedness: question, chunks, answer; are the claims in the chunks? Relevance: question and answer only; is the question addressed?
Never the key	Neither judge sees the key, and each request holds the judge's instruction alone as its system message.
A measurement	Each judge is reduced to two levels and set beside the key grader in an agreement table, with Cohen's kappa.
What comes out	A verdict and a reason per answer per judge, two kappas, and a statement of what the kappas were measured against.

Two judges against the key grader





yes of 22

```
grounded 22

relevant 14


kappa 0.0000

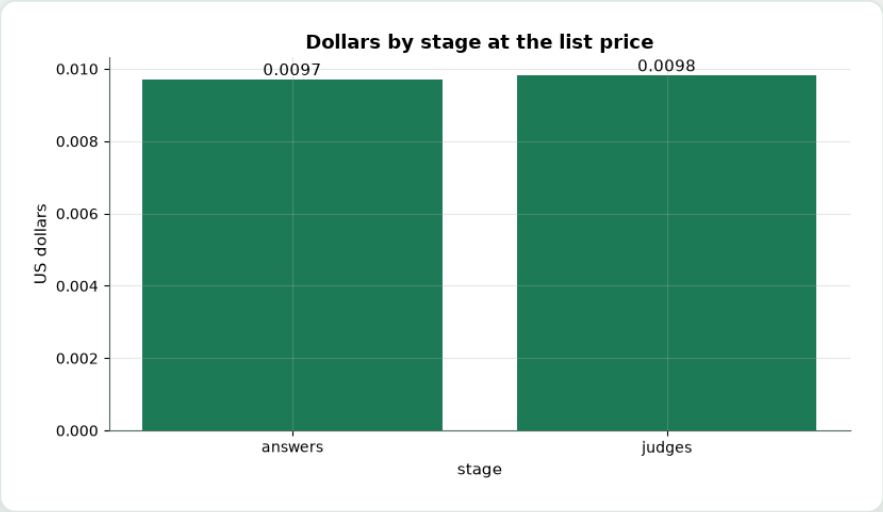
kappa 0.4107
```

The relevance judge's no falls on the NOT IN CORPUS replies, 3 of them the right reply. Day 12's human labels are of day 12's answers: agreement with a person is not measured here, and the report says so.

How it works: what it costs

Tokens only	The API bills the prompt sent and the completion returned; embedding and retrieval run on your processor, unbilled.
From the files	Every saved exchange holds the tokens the API reported, so the cost is computed, never estimated.
cost_table	Stages (answers, judges); tokens times the price per token (per million, divided by 1,000,000); summed; divided by the requests.
A month	A thousand answers each judged twice: one answer plus two verdicts, times a thousand, at a list price read on a stated date.
What comes out	One table, a row per stage, and the two numbers a desk asks for: what this run cost, and what a month of it would.

The bill, and a workbook Excel computes



Excel side by side (tested in Excel)

`=SUM(D2:D19)/ROWS(D2:D19)`	0.6667; Python 0.6667
`=AVERAGE(E2:E19)`	0.6611; Python 0.6611
`=(G4-G5)/(1-G5)`	0.0000 and 0.4107
`=SUMPRODUCT(C2:C3,E2:E3)+SUMPRODUCT(D2:D3,F2:F3)`	0.019539; Python 0.019539

71 requests, 35,844 prompt and 3,251 completion tokens: US$0.019539; a thousand answers, each judged twice, US$0.8064. Workbook: 62 formulas.

gpt-4.1-mini US$0.40 input and US$1.60 output per million tokens (developers.openai.com/api/docs/pricing, read 2026-10-13).

A report that gives no view, and ten commits

The report: 634 words

Scope and settings

Retrieval, with the interval, by document

Answers: retrieval and generation misses

Citations: what the check accepts

Judges, and what is not measured

What the tool does not read; cost

10 commits and the tag course10

71 tests pass (ragkit's 65 and 6 of filing_qa)

one commit for each deliverable that writes a file

assert_extraction_only and opinion_guard are reminders, not proofs.

The AI check

The docstring is the prompt. The retrieval code that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: evaluate_tool keeps only the questions with a place in the folder, so the unanswerable and refused questions leave the denominator; the test returns 1 of 3 rows.

Code that runs is not the same as code that is right.

What you can do now

- Index several documents from pieces and committed embeddings
- Cite every sentence; scope by the date a question states; refuse a view first
- Choose on development questions; open the capstone once
- Tell a retrieval miss from a generation miss; report an interval
- Price the run, write the workbook, and write a report that gives no view

Course 10 complete Implementing RAG